WEEK 6 PRACTICAL QUESTION 1

Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Aim

To implement client-server communication using Named Pipes (FIFOs), where clients send messages to a server and the server processes and responds to them.

Program — Server (server.c)

```
  GNU nano 7.2                                        server.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

#define FIFO1 "client_to_server"
#define FIFO2 "server_to_client"

int main() {
    char message[100];
    char response[150];

    mkfifo(FIFO1, 0666);
    mkfifo(FIFO2, 0666);

    printf("Server started...\n");
    printf("Waiting for client messages...\n");

    while (1) {
        int read_fd = open(FIFO1, O_RDONLY);

        int n = read(read_fd, message, sizeof(message) - 1);
        close(read_fd);

        if (n > 0) {
            message[n] = '\0';

            printf("Client sent: %s\n", message);

            if (strcmp(message, "hello") == 0) {
                strcpy(response, "Hello Client!");
            }
            else if (strcmp(message, "bye") == 0) {
                strcpy(response, "Goodbye Client!");
            }
            else {
                snprintf(response, sizeof(response),
                         "Server received: %s", message);
            }

            int write_fd = open(FIFO2, O_WRONLY);

            write(write_fd, response, strlen(response) + 1);

            close(write_fd);

            printf("Response sent: %s\n", response);
        }
    }

    unlink(FIFO1);
    unlink(FIFO2);

    return 0;
}
```

Program — Client (client.c)

```
 GNU nano 7.2                                client.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define FIFO1 "client_to_server"
#define FIFO2 "server_to_client"

int main() {
    char message[100];
    char response[150];

    printf("Client started.\n");

    while (1) {
        printf("Enter message: ");
        fgets(message, sizeof(message), stdin);

        message[strcspn(message, "\n")] = '\0';

        int write_fd = open(FIFO1, O_WRONLY);

        write(write_fd, message, strlen(message) + 1);

        close(write_fd);

        if (strcmp(message, "bye") == 0) {
            break;
        }

        int read_fd = open(FIFO2, O_RDONLY);

        int n = read(read_fd, response, sizeof(response) - 1);
        close(read_fd);

        if (n > 0) {
            response[n] = '\0';
            printf("Server response: %s\n", response);
        }
    }

    return 0;
}
```

Output

Terminal 1 — Server

```
suraj@DESKTOP-BLT5L3R:~$ mkdir -p ~/Week6
suraj@DESKTOP-BLT5L3R:~$ cd ~/Week6
suraj@DESKTOP-BLT5L3R:~/Week6$ pwd
/home/suraj/Week6
suraj@DESKTOP-BLT5L3R:~/Week6$ nano server.c
suraj@DESKTOP-BLT5L3R:~/Week6$ nano client.c
suraj@DESKTOP-BLT5L3R:~/Week6$ ls
client.c  server.c
suraj@DESKTOP-BLT5L3R:~/Week6$ gcc server.c -o server
cc1: fatal error: server.c: No such file or directory
compilation terminated.
suraj@DESKTOP-BLT5L3R:~/Week6$ mv 'server.c\' server.c
suraj@DESKTOP-BLT5L3R:~/Week6$ ls
client.c  server.c
suraj@DESKTOP-BLT5L3R:~/Week6$ gcc server.c -o server
suraj@DESKTOP-BLT5L3R:~/Week6$ gcc client.c -o client
suraj@DESKTOP-BLT5L3R:~/Week6$ ./server
Server started...
Waiting for client messages...
Client sent: Hello
Response sent: Hello Client!
Client sent: How are you?
Response sent: Server received: How are you?
Client sent: bye
```

Terminal 2 — Client

```
suraj@DESKTOP-BLT5L3R:~$ cd ~/Week6
suraj@DESKTOP-BLT5L3R:~/Week6$ ./client
Client started.
Enter message: hello
Server response: Hello Client!
Enter message: How are you?
Server response: Server received: How are you?
Enter message: bye
suraj@DESKTOP-BLT5L3R:~/Week6$
```

Observation

The client-server application was successfully implemented using Named Pipes (FIFOs). The client sends messages through the client_to_server FIFO, and the server reads and processes the messages. The server sends the processed response back to the client through the server_to_client FIFO. The communication between the client and server was successful. When multiple clients communicate simultaneously, they share the same FIFO, and messages are processed in the order they are written to the pipe. FIFO operations may block when the required reader or writer is not available.

QUESTION 2

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

Aim

To implement POSIX signal handling using SIGINT, SIGTERM, and SIGUSR1, and demonstrate pohow signal handlers respond to asynchronous events.

Program

```
  GNU nano 7.2                          signal_handler.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handle_signal(int sig)
{
    if (sig == SIGINT)
    {
        printf("\nSIGINT received. Interrupt signal handled.\n");
    }
    else if (sig == SIGUSR1)
    {
        printf("\nSIGUSR1 received. User-defined signal handled.\n");
    }
    else if (sig == SIGTERM)
    {
        printf("\nSIGTERM received. Termination signal handled.\n");
    }
}

int main()
{
    signal(SIGINT, handle_signal);
    signal(SIGUSR1, handle_signal);
    signal(SIGTERM, handle_signal);

    printf("Process ID: %d\n", getpid());
    printf("Waiting for signals...\n");
    printf("Press Ctrl+C to send SIGINT.\n");
    printf("Use 'kill -SIGUSR1 PID' to send SIGUSR1.\n");
    printf("Use 'kill -SIGTERM PID' to send SIGTERM.\n");

    while (1)
    {
        pause();
    }

    return 0;
}
```

Output

```
suraj@DESKTOP-BLT5L3R:~$ nano signal_handler.c
saraj@DESKTOP-BLT5L3R:~$ gcc signal_handler.c -o signal_handler
suraj@DESKTOP-BLT5L3R:~$ ./signal_handler
Process ID: 680
Waiting for signals...
Press Ctrl+C to send SIGINT.
Use 'kill -SIGUSR1 PID' to send SIGUSR1.
Use 'kill -SIGTERM PID' to send SIGTERM.
^C
SIGINT received. Interrupt signal handled.
```

Observation

The program registers a signal handler for SIGINT, SIGTERM, and SIGUSR1. When any of these signals is delivered, the operating system asynchronously invokes the corresponding handler.

- **SIGINT** — normally generated using Ctrl+C.
- **SIGTERM** — requests termination of a process.
- **SIGUSR1** — a user-defined signal that can be used for application-specific events.

The pause() system call keeps the process waiting until a signal is received.

Result

The POSIX signal handling program was successfully implemented. The program captured SIGINT, SIGTERM, and SIGUSR1 and demonstrated asynchronous event handling using signal handlers.